

nlp1

December 17, 2025

```
[1]: # =====
# CELL 1: SETUP & LINEAR DATA LOADING (NO FUNCTIONS)
# =====
print(" TASK 1: SETUP & SAFE DATA LOADING (LINEAR MODE)")
print("=" * 60)

# 1. Install & Import
!pip install -q torch transformers[torch] scikit-learn pandas gdown accelerate
↳ nltk sentencepiece

import os
import torch
import pandas as pd
import numpy as np
import gdown
import shutil
import gc
import warnings
import logging
from sklearn.model_selection import train_test_split

# 2. Config
warnings.filterwarnings('ignore')
os.environ["WANDB_DISABLED"] = "true"
logging.getLogger("transformers").setLevel(logging.ERROR)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# 3. Download Data (One by One)
files = {
    'data_eng.csv': '190j4NYhCFyf3wJgsLpPGolm45K-9Idx2',
    'test_eng.csv': '1cMxtHb_bZH1NY19rr7R0izXIzf5xj6Va',
    'data_swa.csv': '1-Mk-PFY2r9PSxac7KJENPq0EcAlKJaij',
    'test_swa.csv': '1dtROGr4go8FFHSMlmYC31brgooUU5ixX',
}
```

```

print("\n Downloading Data...")
for name, file_id in files.items():
    if not os.path.exists(name):
        gdown.download(f'https://drive.google.com/uc?id={file_id}', name,
            ↪quiet=True)

# 4. Load Data Directly (Linear - No Functions)
print("\n Loading Files...")

# A. English Training (Safe Load)
df_eng = pd.read_csv('data_eng.csv', dtype={'id': str})
if 'text' in df_eng.columns:
    df_eng['text'] = df_eng['text'].fillna("").astype(str).str.strip()

# B. Swahili Training (Safe Load)
df_swa = pd.read_csv('data_swa.csv', dtype={'id': str})
if 'text' in df_swa.columns:
    df_swa['text'] = df_swa['text'].fillna("").astype(str).str.strip()

# C. English TEST (CRITICAL: KEEP ALL ROWS)
test_eng = pd.read_csv('test_eng.csv', dtype={'id': str})
if 'text' in test_eng.columns:
    test_eng['text'] = test_eng['text'].fillna("").astype(str).str.strip()

# D. Swahili TEST (CRITICAL: KEEP ALL ROWS)
test_swa = pd.read_csv('test_swa.csv', dtype={'id': str})
if 'text' in test_swa.columns:
    test_swa['text'] = test_swa['text'].fillna("").astype(str).str.strip()

# 5. Filter Training Data Only (Remove empty rows from TRAIN)
# We do NOT touch the Test data rows.
df_eng = df_eng[df_eng['text'].str.len() > 0]
df_swa = df_swa[df_swa['text'].str.len() > 0]

# 6. Create Splits
eng_train_txt, eng_val_txt, eng_train_lbl, eng_val_lbl = train_test_split(
    df_eng['text'].tolist(), df_eng['polarization'].tolist(),
    test_size=0.2, random_state=SEED, stratify=df_eng['polarization']
)
swa_train_txt, swa_val_txt, swa_train_lbl, swa_val_lbl = train_test_split(
    df_swa['text'].tolist(), df_swa['polarization'].tolist(),
    test_size=0.2, random_state=SEED, stratify=df_swa['polarization']
)

print(f" Data Ready.")
print(f" English Train: {len(eng_train_txt)} | Test Rows: {len(test_eng)}")

```

```

print(f"    Swahili Train: {len(swa_train_txt)} | Test Rows: {len(test_swa)}")

# 7. Verification Print
print("\n VERIFYING IDS (Linear Version):")
print("-" * 50)
print(test_eng[['id', 'text']].head())
print("-" * 50)

```

TASK 1: SETUP & SAFE DATA LOADING (LINEAR MODE)

=====

Downloading Data...

Loading Files...

Data Ready.

English Train: 2577 | Test Rows: 160

Swahili Train: 5592 | Test Rows: 349

VERIFYING IDS (Linear Version):

```

-----
                                id \
0  eng_f66ca14d60851371f9720aaf4ccd9b58
1  eng_3a489aa7fed9726aa8d3d4fe74c57efb
2  eng_95770ff547ea5e48b0be00f385986483
3  eng_2048ae6f9aa261c48e6d777bcc5b38bf
4  eng_07781aa88e61e7c0a996abd1e5ea3a20

                                text
0                                God is with Ukraine and Zelensky
1  4 Dems, 2 Republicans Luzerne County Council s...
2  Abuse Survivor Recounts Her Struggles at YWCA ...
3    After Rwanda, another deportation camp disaster
4  Another plea in Trump election interference probe
-----

```

[1]:

[1]:

```

[2]: # =====
# CELL 2: 4-MODEL TRAINING LOOP (10 EPOCHS) & COMPARISON
# =====
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer, EarlyStoppingCallback
from torch.utils.data import Dataset
from sklearn.metrics import f1_score, accuracy_score
import pandas as pd

```

```

import torch
import gc
import numpy as np

print("\n STARTING TRAINING (10 EPOCHS) & COMPARISON...")

# 1. Configuration: The 4 Models to Train
# Format: (Language, Model Name, Save Path, Train Text, Train Label, Val Text,
# ↪Val Label)
ENSEMBLE_CONFIG = [
    # English Models
    ("English", "roberta-base", "./model_eng_1", eng_train_txt, eng_train_lbl,
    ↪eng_val_txt, eng_val_lbl),
    ("English", "bert-base-uncased", "./model_eng_2", eng_train_txt,
    ↪eng_train_lbl, eng_val_txt, eng_val_lbl),

    # Swahili Models
    ("Swahili", "xlm-roberta-base", "./model_swa_1", swa_train_txt,
    ↪swa_train_lbl, swa_val_txt, swa_val_lbl),
    ("Swahili", "bert-base-multilingual-cased", "./model_swa_2", swa_train_txt,
    ↪swa_train_lbl, swa_val_txt, swa_val_lbl),
]

# Store results here for the final table
comparison_results = []

# 2. Standard Dataset Class (Required for Hugging Face)
class PolarizationDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self): return len(self.texts)

    def __getitem__(self, idx):
        enc = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding='max_length',
            max_length=self.max_len,
            return_tensors='pt'
        )
        return {
            'input_ids': enc['input_ids'].flatten(),
            'attention_mask': enc['attention_mask'].flatten(),

```

```

        'labels': torch.tensor(self.labels[idx], dtype=torch.long)
    }

# 3. Metric Function (To calculate F1 Score)
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    return {
        'f1_macro': f1_score(p.label_ids, preds, average='macro'),
        'accuracy': accuracy_score(p.label_ids, preds)
    }

# 4. Main Training Loop
for lang, model_name, save_path, tr_txt, tr_lbl, val_txt, val_lbl in in
    ↪ENSEMBLE_CONFIG:
        print(f"\n" + "="*60)
        print(f"  TRAIN START: {lang} | Model: {model_name}")
        print(f"="*60)

        # Clear Memory
        torch.cuda.empty_cache(); gc.collect()

        try:
            # Load Model & Tokenizer
            tokenizer = AutoTokenizer.from_pretrained(model_name)
            model = AutoModelForSequenceClassification.from_pretrained(model_name, in
            ↪num_labels=2).to(DEVICE)

            # Training Settings (10 Epochs)
            args = TrainingArguments(
                output_dir=f"results_{save_path}",
                num_train_epochs=10,                # Maximum learning
                per_device_train_batch_size=32,
                learning_rate=2e-5,
                fp16=True,                          # Faster training on GPU
                eval_strategy="epoch",
                save_strategy="epoch",
                load_best_model_at_end=True,          # Keeps the smartest version
                metric_for_best_model="f1_macro",
                save_total_limit=1,
                report_to="none",
                logging_steps=50
            )

            # Initialize Trainer
            trainer = Trainer(
                model=model, args=args,
                train_dataset=PolarizationDataset(tr_txt, tr_lbl, tokenizer),

```

```

        eval_dataset=PolarizationDataset(val_txt, val_lbl, tokenizer),
        compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(early_stopping_patience=3)] # Stop
        ↪if it doesn't improve for 3 epochs
    )

    # Train
    trainer.train()

    # Get Final Score
    eval_metrics = trainer.evaluate()
    final_f1 = eval_metrics['eval_f1_macro']
    comparison_results.append({
        "Language": lang,
        "Model": model_name,
        "F1 Score (Macro)": final_f1
    })

    # Save Model for Prediction (Cell 3)
    trainer.save_model(save_path)
    tokenizer.save_pretrained(save_path)
    print(f" Saved: {save_path} (F1: {final_f1:.4f})")

except Exception as e:
    print(f"Error with {model_name}: {e}")

# 5. Print Final Leaderboard
print("\n" + "="*60)
print(" MODEL LEADERBOARD (Who is smartest?)")
print("="*60)
results_df = pd.DataFrame(comparison_results)
results_df = results_df.sort_values(by=['Language', 'F1 Score (Macro)'],
    ↪ascending=[True, False])
print(results_df.to_string(index=False))
print("="*60)

```

STARTING TRAINING (10 EPOCHS) & COMPARISON...

```

=====
TRAIN START: English | Model: roberta-base
=====

tokenizer_config.json:  0%|          | 0.00/25.0 [00:00<?, ?B/s]
config.json:           0%|          | 0.00/481 [00:00<?, ?B/s]
vocab.json:            0%|          | 0.00/899k [00:00<?, ?B/s]

```

merges.txt: 0%| | 0.00/456k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.36M [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/499M [00:00<?, ?B/s]

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized:

['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Saved: ./model_eng_1 (F1: 0.8129)

=====

TRAIN START: English | Model: bert-base-uncased

=====

tokenizer_config.json: 0%| | 0.00/48.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/570 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/232k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/466k [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/440M [00:00<?, ?B/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized:

['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Saved: ./model_eng_2 (F1: 0.7715)

=====

TRAIN START: Swahili | Model: xlm-roberta-base

=====

tokenizer_config.json: 0%| | 0.00/25.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/615 [00:00<?, ?B/s]

sentencepiece.bpe.model: 0%| | 0.00/5.07M [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/9.10M [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/1.12G [00:00<?, ?B/s]

Some weights of XLMRobertaForSequenceClassification were not initialized from the model checkpoint at xlm-roberta-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Saved: ./model_swa_1 (F1: 0.7977)

```
=====
  TRAIN START: Swahili | Model: bert-base-multilingual-cased
=====
```

tokenizer_config.json: 0%| | 0.00/49.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/625 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/996k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.96M [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/714M [00:00<?, ?B/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-multilingual-cased and are newly initialized:
['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Saved: ./model_swa_2 (F1: 0.7920)

```
=====
  MODEL LEADERBOARD (Who is smartest?)
=====
```

Language	Model	F1 Score (Macro)
English	roberta-base	0.812896
English	bert-base-uncased	0.771450
Swahili	xlm-roberta-base	0.797663
Swahili	bert-base-multilingual-cased	0.791987

[2]:

[2]:


```

[4]: # =====
# CELL 3: PREDICTION & GENERATE SUBMISSION (TEXT IN MIDDLE)
# =====
print("\n PREDICTING & GENERATING ZIP (WITH TEXT IN MIDDLE)...")

import pandas as pd
import numpy as np
import os
import shutil
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from tqdm.auto import tqdm
from datetime import datetime

# 1. Setup
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
submission_folder = "subtask_1"

# 2. Run English Predictions (Linear - No Functions)
print("\n Predicting English...")
eng_probs = None
eng_models = ["/model_eng_1", "/model_eng_2"]

for path in eng_models:
    if os.path.exists(path):
        print(f"    -> Loading {path}...")
        tokenizer = AutoTokenizer.from_pretrained(path)
        model = AutoModelForSequenceClassification.from_pretrained(path).
        ↪to(DEVICE)
        model.eval()

        # Batch Predict
        current_probs = []
        texts = test_eng['text'].tolist()
        for i in range(0, len(texts), 32):
            batch = texts[i:i+32]
            inputs = tokenizer(batch, padding=True, truncation=True,
            ↪max_length=128, return_tensors="pt").to(DEVICE)
            with torch.no_grad():
                outputs = model(**inputs)
                current_probs.extend(F.softmax(outputs.logits, dim=-1).cpu().
            ↪numpy())

        if eng_probs is None: eng_probs = np.array(current_probs)
        else: eng_probs += np.array(current_probs)

```

```

# Final Argmax for English
eng_preds = np.argmax(eng_probs, axis=1)

# 3. Run Swahili Predictions (Linear - No Functions)
print("\n Predicting Swahili...")
swa_probs = None
swa_models = ["/model_swa_1", "/model_swa_2"]

for path in swa_models:
    if os.path.exists(path):
        print(f"    -> Loading {path}...")
        tokenizer = AutoTokenizer.from_pretrained(path)
        model = AutoModelForSequenceClassification.from_pretrained(path).
        ↪to(DEVICE)
        model.eval()

        # Batch Predict
        current_probs = []
        texts = test_swa['text'].tolist()
        for i in range(0, len(texts), 32):
            batch = texts[i:i+32]
            inputs = tokenizer(batch, padding=True, truncation=True,
            ↪max_length=128, return_tensors="pt").to(DEVICE)
            with torch.no_grad():
                outputs = model(**inputs)
                current_probs.extend(F.softmax(outputs.logits, dim=-1).cpu().
            ↪numpy())

        if swa_probs is None: swa_probs = np.array(current_probs)
        else: swa_probs += np.array(current_probs)

# Final Argmax for Swahili
swa_preds = np.argmax(swa_probs, axis=1)

# 4. Create Submission Files (With Text in Middle)
if os.path.exists(submission_folder): shutil.rmtree(submission_folder)
os.makedirs(submission_folder)

# English File: id, text, polarization
df_eng_sub = test_eng.copy()
df_eng_sub['polarization'] = eng_preds
# REORDER COLUMNS: ID -> TEXT -> POLARIZATION
df_eng_sub = df_eng_sub[['id', 'text', 'polarization']]
df_eng_sub.to_csv(f"{submission_folder}/pred_eng.csv", index=False)

```

```

# Swahili File: id, text, polarization
df_swa_sub = test_swa.copy()
df_swa_sub['polarization'] = swa_preds
# REORDER COLUMNS: ID -> TEXT -> POLARIZATION
df_swa_sub = df_swa_sub[['id', 'text', 'polarization']]
df_swa_sub.to_csv(f"{submission_folder}/pred_swa.csv", index=False)

# 5. Zip
timestamp = datetime.now().strftime('%H%M%S')
zip_filename = f"submission_task1_WITH_TEXT_{timestamp}"
shutil.make_archive(zip_filename, 'zip', root_dir='.',
    ↪base_dir=submission_folder)

print("\n" + "="*60)
print(f" SUBMISSION READY: {zip_filename}.zip")
print(f" Columns: id, text, polarization")
print(f" Text is in the middle.")
print("="*60)

```

PREDICTING & GENERATING ZIP (WITH TEXT IN MIDDLE)...

Predicting English...

-> Loading ./model_eng_1...

-> Loading ./model_eng_2...

Predicting Swahili...

-> Loading ./model_swa_1...

The tokenizer you are loading from './model_swa_1' with an incorrect regex pattern: <https://huggingface.co/mistralai/Mistral-Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e>. This will lead to incorrect tokenization. You should set the `fix\_mistral\_regex=True` flag when loading this tokenizer to fix this issue.

The tokenizer you are loading from './model_swa_2' with an incorrect regex pattern: <https://huggingface.co/mistralai/Mistral-Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e>. This will lead to incorrect tokenization. You should set the `fix\_mistral\_regex=True` flag when loading this tokenizer to fix this issue.

-> Loading ./model_swa_2...

```

=====
SUBMISSION READY: submission_task1_WITH_TEXT_125721.zip
Columns: id, text, polarization
Text is in the middle.
=====

```

[3] :

[3] :